Leveraging Defects Lifecycle for Labeling Defective Classes

Agenda

- Introduction
  - Context/ Definitions/ Aim
- RQ1
  - Rationale/ Design/ Results/ Discussion
- RQ2
- RQ3
- RQ4
- Conclusions
- Future Work

Introduction - Context

- Defect prediction aims at identifying software artifacts that are likely to exhibit a defect.
- The main purpose of defect prediction is to reduce the cost of testing and code review, by letting developers focus on specific artifacts.
- Ultimately, the identification of likely defect prone files helps in prioritizing testing and analysis activities

Introduction - Context

- This work focuses on the creation of datasets used for defect prediction. Specifically, automated methods for labeling classes as defective or not

Labeling Classes as Defective or Not

- SZZ
- Earliest AV in JIRA

Defects Lifecycle

https://issues.apache.org/jira/browse/QPID -4462

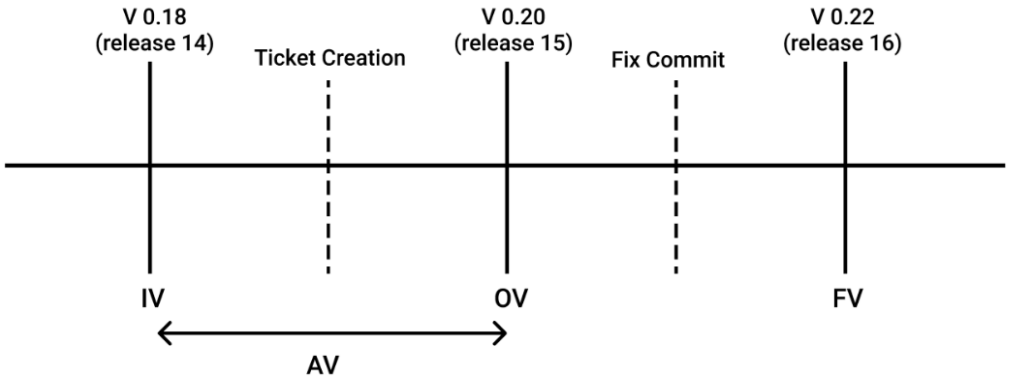


Figure 1.1: Example of the life-cycle of a defect: Introduction Version (IV), Opening Version (OV), Fixed Version (FV), and Affected Versions (AV).

SZZ

- SZZ : Process to identify when a defect has been actually introduced in a software project
  - Exploits the versioning system annotation mechanism (e.g. git blame)
- Given a fix commit, the algorithm determines, for the source code lines that have been changed, when they have last been changed before such a fix
- Limitations
  - Not perfectly accurate
  - Cannot find bugs of the regression type
  - Cannot find bugs where only the addition of code was used to fix the bug

Introduction - Aim

- Investigate the extent to which the AV-related information is available in open-source project and whether it is reliable to be used in labeling classes as defective or not for building defect datasets
- Propose, evaluate, and compare to SZZ *a new method for labeling defective classes* which is based on the idea that defects, even of different projects, have a stable life-cycle

Introduction - Research Questions

- RQ1: Is the AV available and consistent
- RQ2: Do methods have different accuracy in AV labeling?
- RQ3: Do methods have different accuracy in classes labeling?
- RQ4: Do methods lead to different identification of important features?

RQ1: Is the AV available and consistent - Rationale

- We want to know whether the AV is available, i.e., did Jira developers enter AV into the defects ticket information
- If so, is the AV consistent, i.e., is the oldest AV not after the OV?

RQ1: Is the AV available and consistent - Design

- Independent variable: the defect
- Dependent variable: percentage of available and available & consistent AV

RQ1: Is the AV available and consistent - Design

- Used 212 Apache Projects that were linked with Git and Jira
- Selected issue reports where

Type == “Bug” AND (status == “Closed” OR status == “Resolved”) AND Resolution == “Fixed”

- Excluded defects that did not have a Git related fix commit
- Excluded defects that are not post-release

RQ1: Is the AV available and consistent - Results

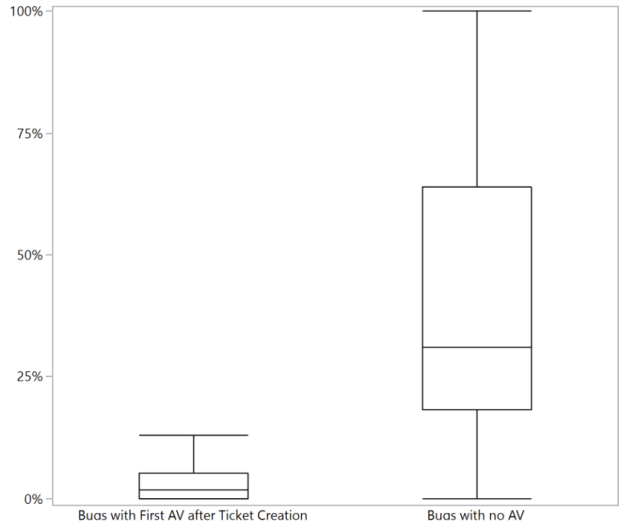


Figure 4.1: Distribution of 212 Apache projects having a specific proportion of bugs with an unreliable AV (left side) or without the AV (right side).

RQ1: Is the AV available and consistent - Discussion

- Most of the projects have more than 25% of defects whose issue report contained no AV
- The total number of closed bugs linked with git in the 212 Apache projects is 125,860
- 63,539 bugs , i.e., 51%, resulted in not having or having an unreliable AV
- Most of the bugs of Apache project we can't use the AV and hence we frequently need an automated method for labeling classes

RQ2: Do methods have different accuracy in AV labeling? - Rationale

- Defects may contain a stable lifecycle:
  - It takes a certain number of versions to find the bug (OV)
  - It takes a certain number of versions to fix the bug (FV)
- Given the results from RQ1, we want to know if using a defect's lifecycle information is accurate in labeling the AV compared to existing solutions, SZZ
- We want to determine if a version is defective by a given bug

RQ2: Do methods have different accuracy in AV labeling? - Design

- Project Selection:
  - Projects with greater than 50% of available and consistent AV
  - Projects with at least 100 bugs that are linked to a Git fix commit and contained available and consistent AV
  - Projects with at least 6 versions
  - 76 Apache Jira projects remaining

RQ2: Do methods have different accuracy in AV labeling? - Design

- Defect's life-cycle:
  - Intuition: There will be a proportional number of versions to find the defect and to fix the defect
  - P = (FV- IV) / (FV - OV)
  - Predicted IV = FV- (FV - OV) * P
  - Variations of computing P:
    - Cold Start: Compute P as the median among the 75 other projects
    - Increment: Compute P as the average among the defects fixed in previous versions
    - Moving Window: Compute P as the average among the last 1% of fixed defects

RQ2: Do methods have different accuracy in AV labeling? - Design

- Independent Variables: the methods
  - Simple: Assume the IV is the same as OV
  - SZZ:
    - SZZ_B (Basic), SZZ_U (Unleashed), SZZ_RA (Refactoring Aware)
  - Proportional:
    - Proportional_ColdStart, Proportional_Increment, Proportional_MovingWindow
  - +: each SZZ in combination with Simple

RQ2: Do methods have different accuracy in AV labeling? - Design

- Dependent variables:
  - True Positive(TP): The version is measured to be defective by the defect and is labeled to be defective.
  - True Negative(TN): The version is measured to be non-defective by the defect and is labeled to be non-defective.
  - False Positive(FP): The version is measured to be non-defective by the defect and is labeled to be defective.
  - False Negative(FN): The version is measured to be defective by the defect and is labeled to be non-defective.

RQ2: Do methods have different accuracy in AV labeling? - Design

- Dependent variables:
  - Precision: TP / (TP + FP)
  - Recall: TP / (TP + FN)
  - F1: (2 * Precision * Recall) / (Precision + Recall)
  - Kappa: Measures performance against random guessing
  - Matthews Correlation Coefficient: Measures quality of a two class classification

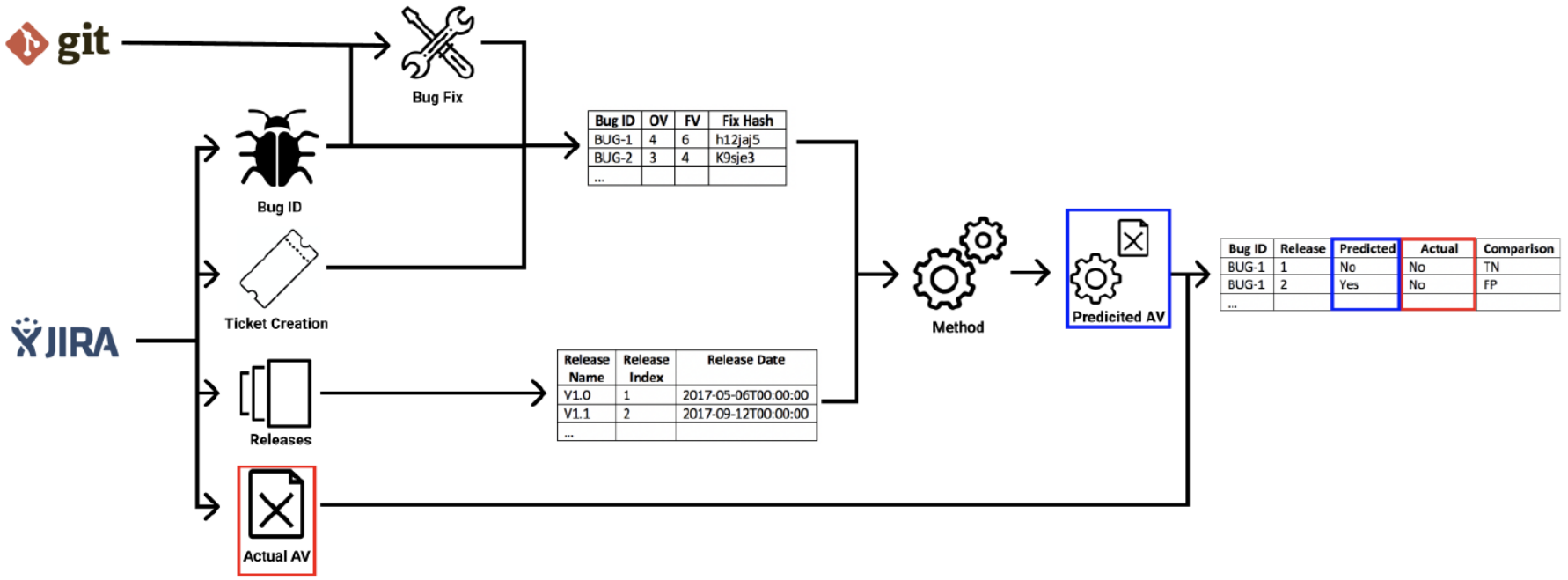


Figure 3.1: General overview to compute predicted AV in RQ2 for each method.

RQ2: Do methods have different accuracy in AV labeling? - Results

- The Proportion methods have a higher Precision and composite accuracy (F1, MCC, and Kappa) than all SZZ
- Simple has a higher Precision and composite accuracy than all SZZ methods
- SZZ_U has the highest Recall than all other methods
- SZZ_B+ has the highest Precision and the highest composite accuracy than any other SZZ method.
- There is no single dominant method among the Proportion methods.

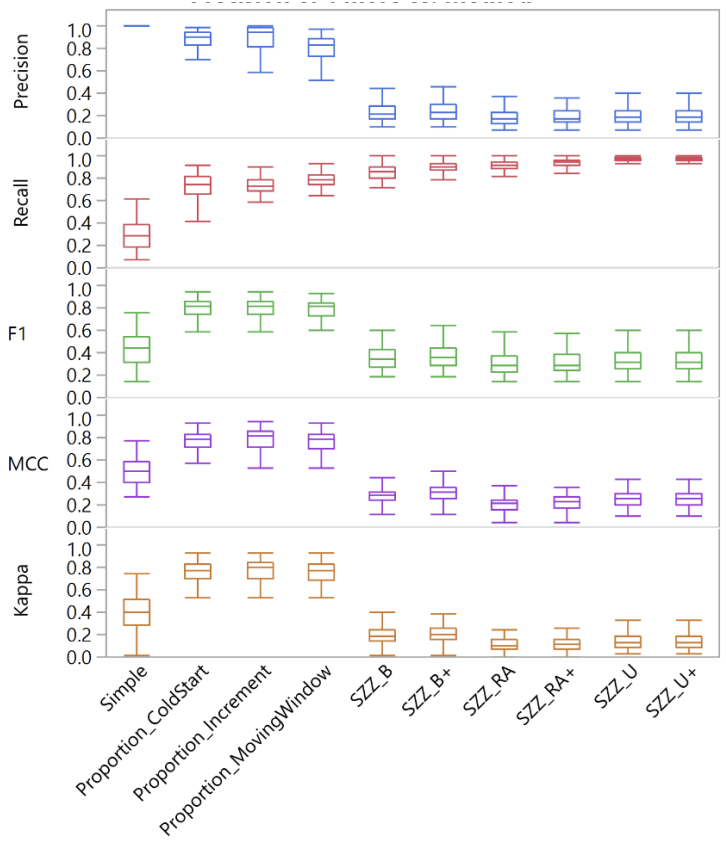


Figure 4.2: Distribution, across 76 Apache projects, of Precision, Recall, F1, MCC, and Kappa, of different methods in labeling AV.

RQ2: Do methods have different accuracy in AV labeling? - Results

Version	STDV
IV	38.36
OV	40.17
FV	41.85
P	5.43

Table 4.1: Variation, in terms of standard deviation, of IV, OV, FV, and P across bugs of 76 Apache projects.

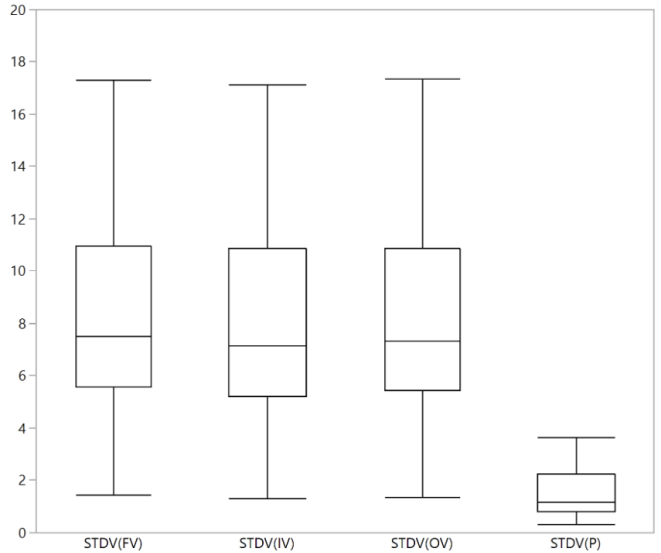


Figure 4.4: Distribution of standard deviation of IV, OV, FV, and P, across 76 Apache projects.

RQ2: Do methods have different accuracy in AV labeling? - Discussion

- P is pretty stable across bugs of different projects especially when compared to IV, OV and FV
- STDV is much higher across projects than within the same project
  - About 5 across projects and 2 within projects
- Bugs shares the same lifecycle in terms of proportion of number of versions between its fix and its discovery, especially within the same project

RQ2: Do methods have different accuracy in AV labeling? - Results

Table 4.2: Relative increment in the accuracy of labeling AV, by combining Simple with a SZZ method, over SZZ, in average across bugs of 76 Apache projects.

	SZZ_B	SZZ_B+	Gain on SZZ_B	SZZ_RA	SZZ_RA+	Gain on SZZ_RA	SZZ_U	SZZ_U+	Gain on SZZ_U
Precision	0.252	0.263	4%	0.206	0.21	2%	0.223	0.224	0%
Recall	0.848	0.894	5%	0.915	0.938	2%	0.97	0.971	0%
F1	0.369	0.387	5%	0.318	0.326	2%	0.345	0.345	0%
MCC	0.296	0.343	14%	0.213	0.229	7%	0.271	0.272	0%
Kappa	0.199	0.222	10%	0.122	0.131	7%	0.157	0.157	0%

RQ2: Do methods have different accuracy in AV labeling? - Discussion

- Low increases from SZZ_X to SZZ_X+
- SZZ_B had the largest increase in MCC (+14%)
- The p-value of the paired Wilcoxon Signed Rank test resulted less than 0.01 for Precision, Recall, F1, MCC, and Kappa in all three SZZ methods
- Therefore, there is a significant difference in the accuracy of SZZ methods with the addition of bugs' information over that SZZ method.

RQ3: Do methods have different accuracy in classes labeling? - Rationale

- It is important to understand which methods provides the most accurate defect prediction dataset
- We want to know, in the within-project across-version class-level context, if a specific class in a specific version is labeled to be defective or not.

RQ3: Do methods have different accuracy in classes labeling? - Design

- Independent variables: the same methods as RQ2
- Dependent variables: the same as RQ2 with the only difference that the unit upon which the accuracy is computed is the defectiveness of a class in a version
  - True Positive(TP): The class in a version is measured to be defective and is labeled to be defective.
  - False Negative(FN): The class in a version is measured to be defective and is labeled to be non-defective.
  - True Negative(TN): The class in a version is measured to be non-defective and is labeled to be non-defective.
  - False Positive(FP): The class in a version is measured to be non-defective and is labeled to be defective.

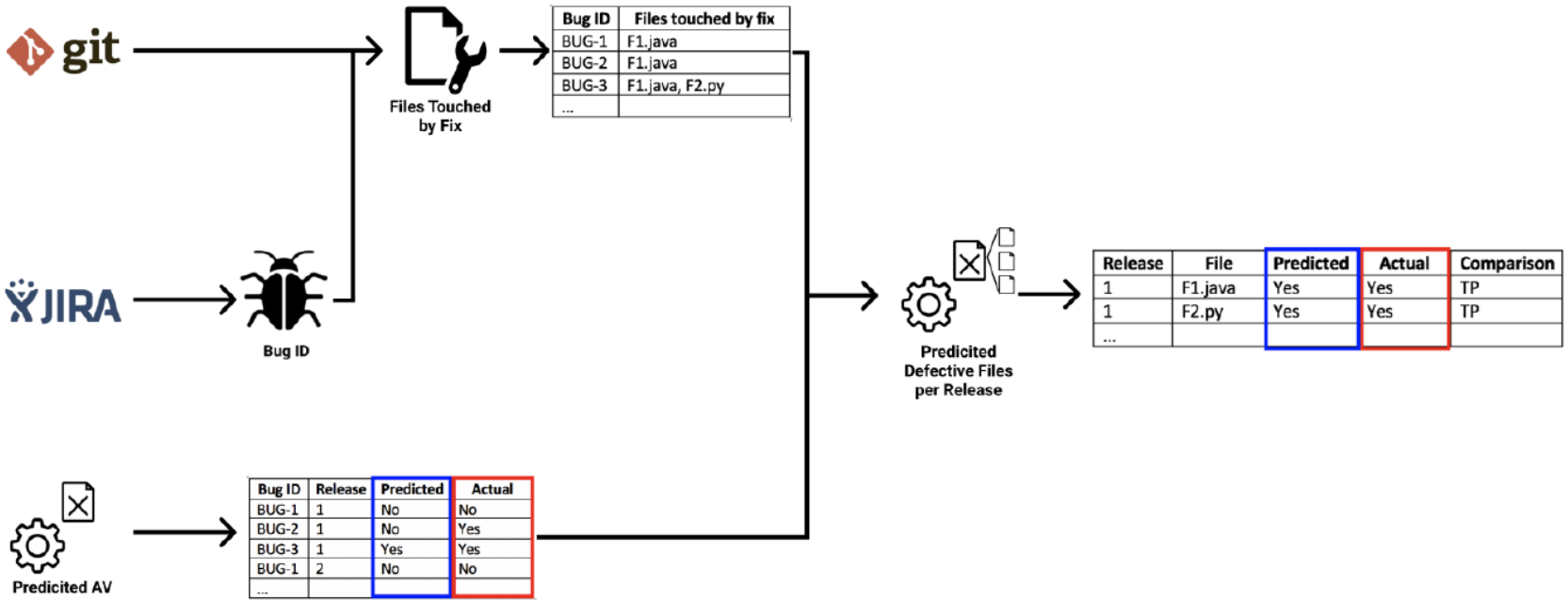


Figure 3.2: General overview to compute predicted defective files for each release within a project in RQ3 for each method.

RQ3: Do methods have different accuracy in classes labeling? - Results

- As in RQ2, the Proportion methods have a higher Precision and composite accuracy (F1, MCC, and Kappa than all SZZ methods
- As in RQ2, SZZ_U has the highest Recall than all other methods.
- As in RQ2, SZZ_B+ has a highest Precision and lower Recall than any other SZZ method.
- Differently from RQ2, SZZ_B+ has a higher composite accuracy than Simple and any other SZZ method.
- Differently than in RQ2, the Proportion_MovingWindow method dominates all methods on all composite accurac

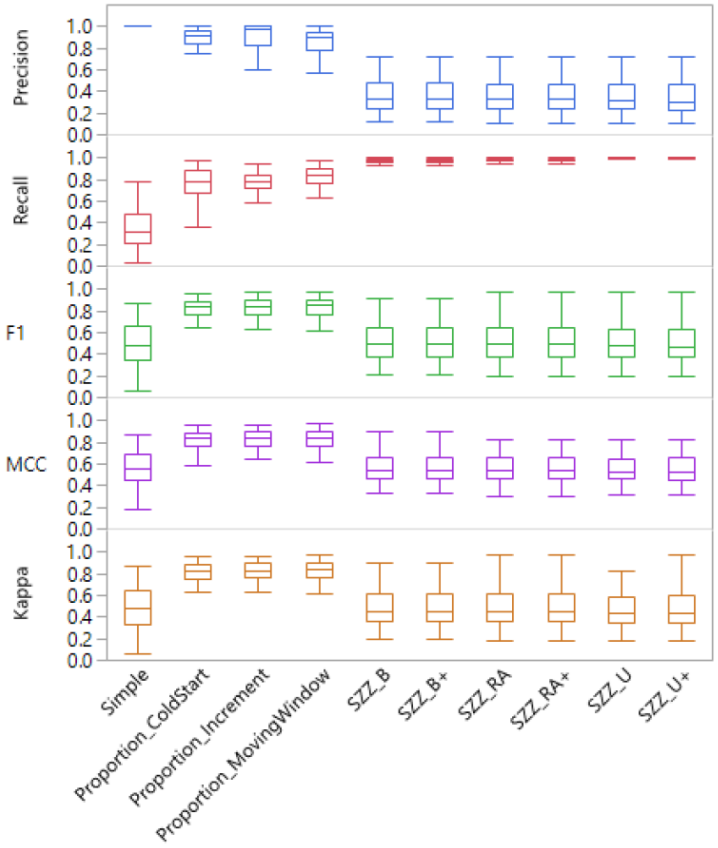


Figure 4.5: Distribution, across 76 Apache projects, of Precision, Recall, F1, MCC, and Kappa, of different methods in labeling defective classes.

RQ3: Do methods have different accuracy in classes labeling? - Discussion

- Regarding comparing the accuracy in class labeling of SZZ methods with bugs' information over that SZZ method, our results show less than 1% of improvement
- All methods are more accurate in labeling classes than AV on all accuracy metrics
  - By comparing the median accuracy (across methods and datasets) we see an increase in labeling classes over AV of 13% in Precision, 5% in Recall, 16% in F1, 27% in MCC and 39% in Kappa

RQ4: Do methods lead to different identification of important features? - Rationale

- In this research question, we investigate the accuracy of methods in identifying important features affecting class defectiveness
- Past research reports that it is important to understand the most important metrics for identifying class defectiveness, developers can avoid the pitfalls that show high association with the appearance of defective classes
- This question is a twofold:
  - Which features are selected to be used by the classifiers
  - Is the feature highly correlated with the predicted variable

RQ4: Do methods lead to different identification of important features? - Design: Features

Table 3.1: Defect prediction features.

Metric	Description
Size	Lines of code(LOC)
LOC Touched	Sum over revisions of LOC added + deleted
NR	Number of revisions
Nfix	Number of bug fixes
Nauth	Number of authors
LOC Added	Sum over revisions of LOC added
MAX LOC Added	Maximum over revisions of LOC added
AVG LOC Added	Average LOC added per revision
Churn	Sum over revisions of added - deleted LOC
Max Churn	Maximum churn over revisions
Average Churn	Average churn over revisions
Change Set Size	Number of files committed together
Max Change Set	Maximum change set size over revisions
Average Change Set	Average change set size over revisions
Age	Age of Release
Weighted Age	Age of Release weighted by LOC touched

RQ4: Do methods lead to different identification of important features? - Design

- Independent variables: the same methods presented in RQ2
- Dependent variables
  - Feature selection: we compare the features selected by using a dataset developed by a method vs. the features selected by using a dataset of the actual methods
    - TP/TN/FP/FN, Precision, Recall, F1, MCC, Kappa
  - Correlation: we compare the correlation measured on a dataset developed by a method versus the correlation measured on the actual dataset
    - Standard Mean Relative Error: | Predicted- Actual | / Actual

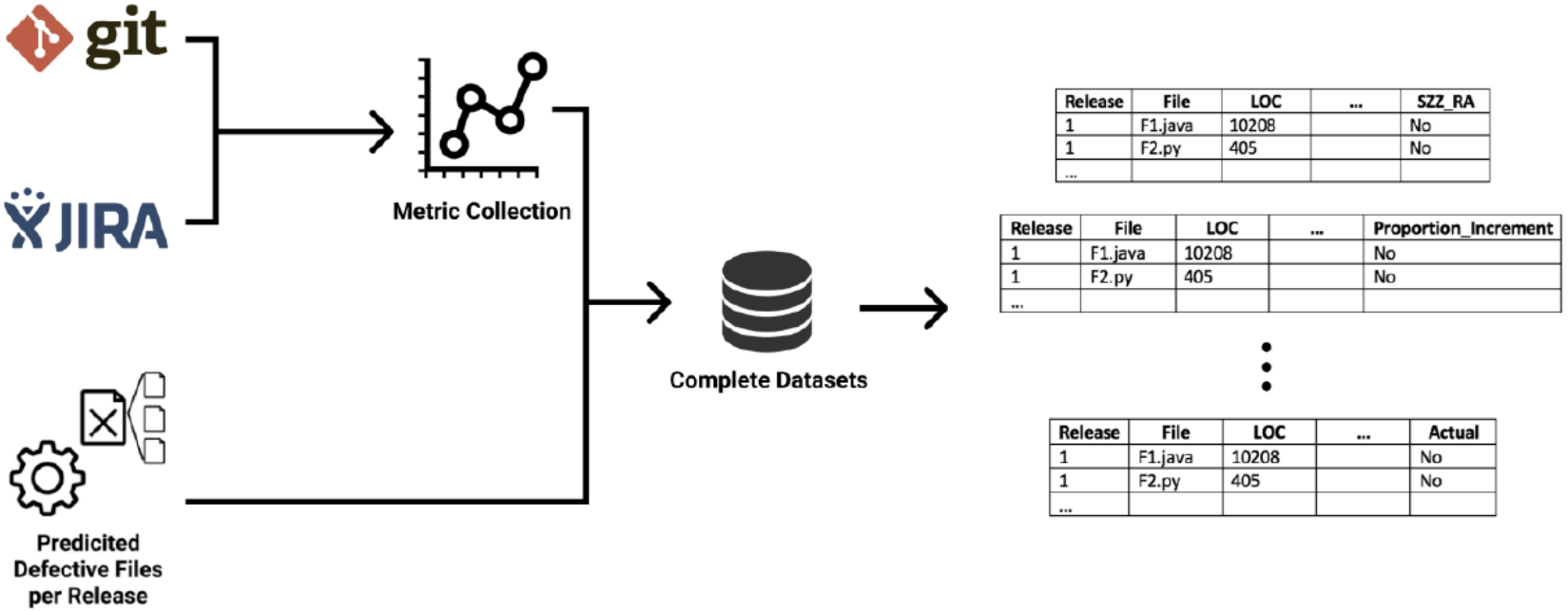


Figure 3.3: General overview of the process of creating the Complete datasets for each project and method.

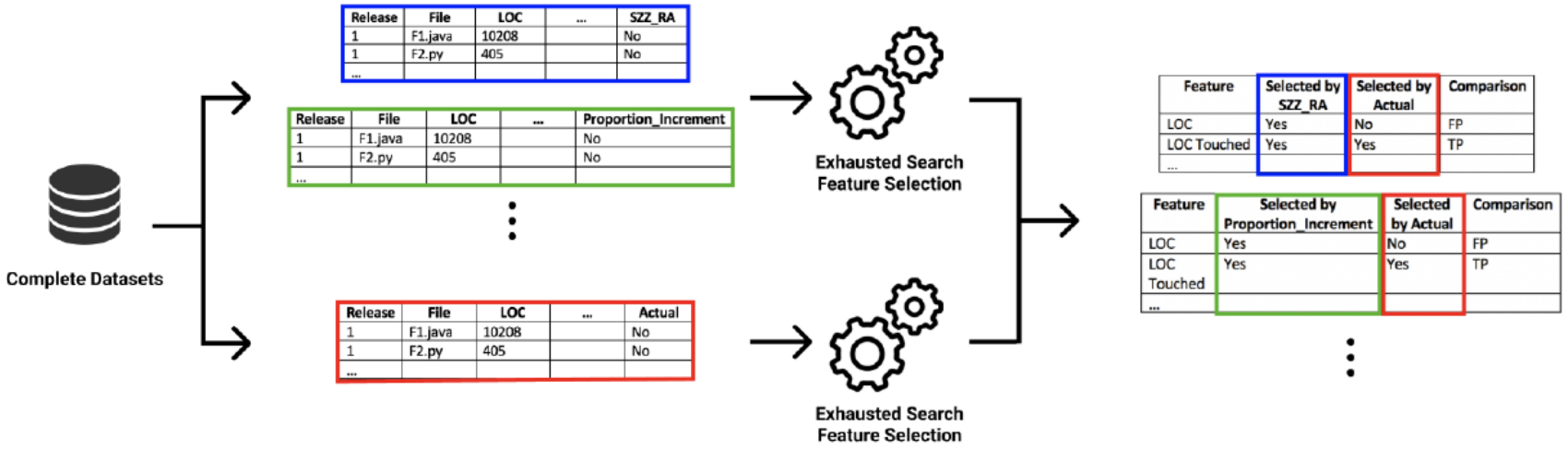


Figure 3.4: General overview of the process of feature selection for each release of each project.

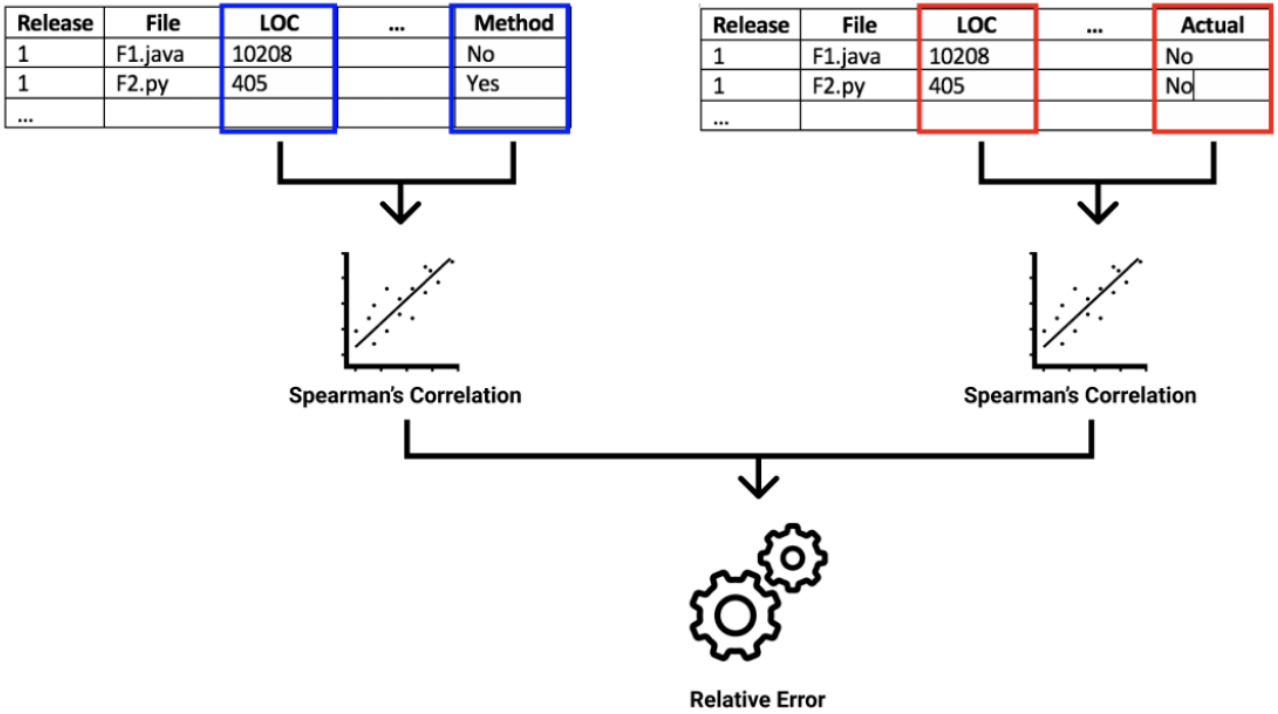


Figure 3.5: General overview of the process of correlation in RQ4 computed for each project, release, method, and feature.

RQ4: Do methods lead to different identification of important features? - Results

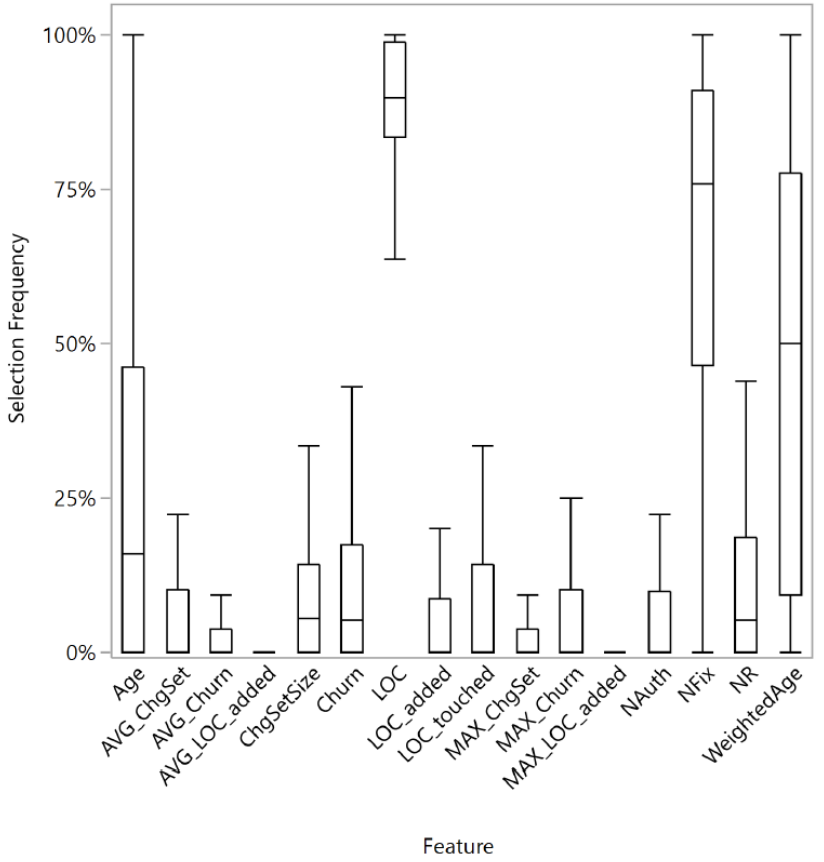


Figure 4.6: Distribution among datasets of selection frequency of each feature.

RQ4: Do methods lead to different identification of important features? - Results

- The Proportion methods have a higher accuracy (in all five metrics) than all SZZ methods. Specifically, the proportion methods are the only methods having a perfect median Precision and Recall.
- Differently from RQ2 and RQ3, there is not much difference in the accuracy of different SZZ methods.
- There is less than 1% of improvement in accuracy of feature selection between the SZZ_X and SZZ_X+ methods

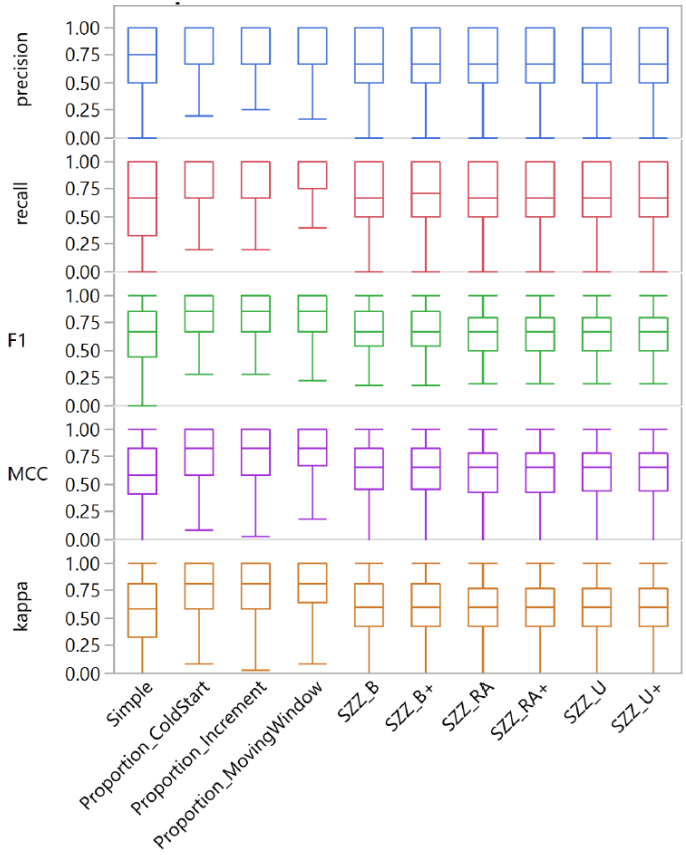


Figure 4.7: Distribution, across releases and 76 Apache projects, of Precision, Recall, F1, MCC, and Kappa, of different methods in selecting features.

RQ4: Do methods lead to different identification of important features? - Results

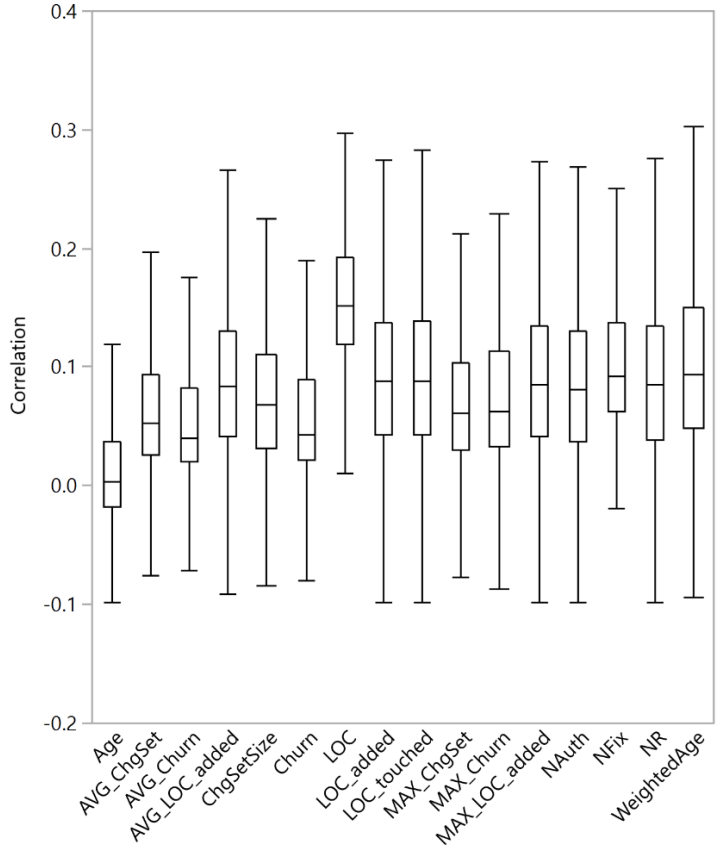


Figure 4.8: Distribution among datasets of correlation of each feature to class defectiveness.

RQ4: Do methods lead to different identification of important features? - Results

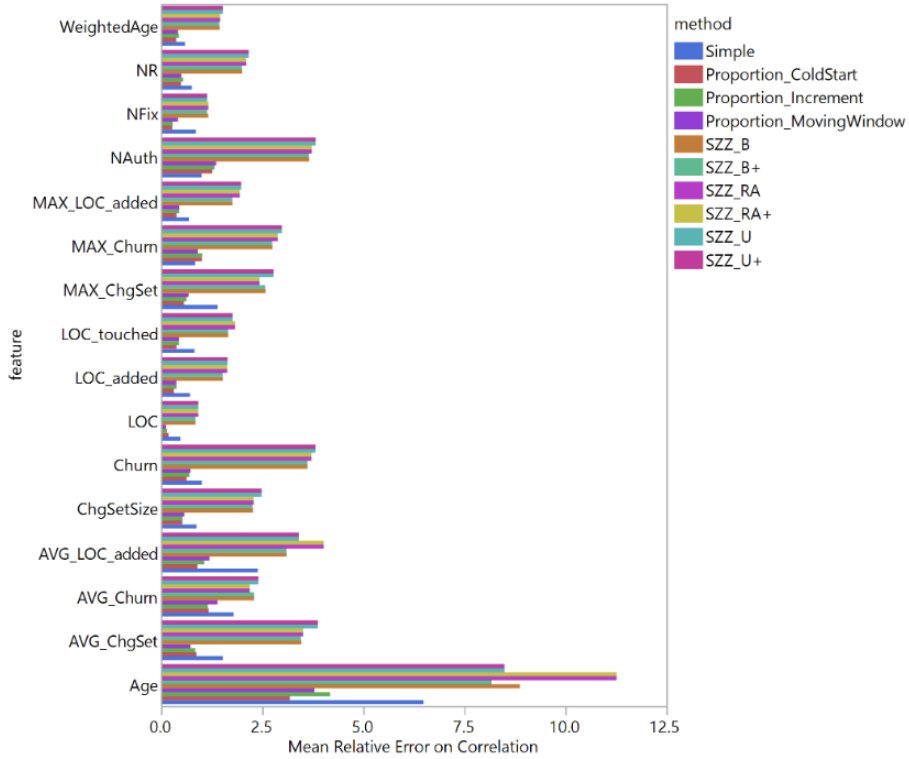


Figure 4.9: Mean relative error of correlation, for each feature, of each method.

RQ4: Do methods lead to different identification of important features? - Discussion

- The Proportion methods have a higher accuracy than all SZZ methods. Specifically, the proportion methods have an error that is about 25% of the SZZ methods' error.
- Differently from RQ2 and RQ3, the Simple method is more accurate than all SZZ methods. Specifically, the Simple method has an error that is about half of the SZZ methods' error.
- There is less than 1% of improvement in accuracy of feature selection between the SZZ_X and SZZ_X+ methods

Conclusion

- Results related to 212 open-source projects from the Apache ecosystem, featuring a total of about 125,000 defects, show that the AV cannot be used in the majority (51%) of defects
- Results related to 76 open-source projects from the Apache ecosystem, showed that the proposed method for labeling defective classes is, in average among projects and bugs, more accurate, in terms of Precision, Kappa, F1, and MCC than all previously proposed SZZ methods
- The improvement in accuracy from combining SZZ with defects life-cycle information is statistically significant but practically irrelevant (< 1%)
- Selecting features and observing correlations information are both, overall and in average, significantly more accurate via bugs' life-cycle than any SZZ method

Future Work

- Analyzing other bug introducing commits in SZZ
- Analyzing the accuracy of affect versions labeled by JIRA developers
- Replication in context of JIT
- Finer combination of Proportion and SZZ methods